

# ShipNet — Guía de trabajo: Daniel Villarroel

Rol: Base de datos · Sprint 1 (11/09 - 15/09/2026) · Programación Web, UPDS Cochabamba

## 1. Cómo subir tus cambios al repositorio

El proyecto vive en [github.com/angelops1998/shipnet](https://github.com/angelops1998/shipnet). Nadie sube cambios directo a la rama **main**: cada tarea (issue) se trabaja en su propia rama y se integra mediante un **Pull Request** (PR) que otro integrante revisa antes de aprobar.

### 1.1 Configuración inicial (solo la primera vez)

1. Acepta la invitación como colaborador que te llegó por correo (o entra a [github.com/angelops1998/shipnet/invitations](https://github.com/angelops1998/shipnet/invitations)).

2. Clona el repositorio en tu computadora:

```
git clone https://github.com/angelops1998/shipnet.git
cd shipnet
```

3. Crea tu archivo de configuración local a partir de la plantilla. Este archivo **no se sube** a GitHub (está en el `.gitignore`), así que cada uno pone su propia contraseña de MySQL:

```
cp appsettings.Development.json.example appsettings.Development.json
```

Luego abre `appsettings.Development.json` y reemplaza **TU\_PASSWORD\_AQUI** por la contraseña de tu MySQL local.

4. Verifica que el proyecto compile:

```
dotnet build
```

### 1.2 Flujo para cada issue

Repite estos pasos por cada issue que tomes. Usa el número de issue (SN-XX) en el nombre de la rama para que sea fácil identificarla.

1. Ponte en **main** y trae lo último que subió el equipo:

```
git checkout main
git pull
```

2. Crea una rama nueva para tu issue (ejemplo con SN-01):

```
git checkout -b feature/sn-01-modelo-er
```

3. Trabaja normalmente. Cuando tengas un avance que valga la pena guardar, revisa qué cambió y haz un commit con un mensaje claro:

```
git status
git add .
git commit -m "SN-01: diagrama ER con entidades y relaciones"
```

Antes de **git add .** mira la lista de **git status**: si aparece `appsettings.Development.json`, `bin/` u `obj/`, algo está mal con el `.gitignore` — avisa antes de commitear.

4. Sube tu rama a GitHub (la primera vez con **-u**; las siguientes basta **git push**):

```
git push -u origin feature/sn-01-modelo-er
```

### 1.3 Crear el Pull Request

1. Entra a **github.com/angelops1998/shipnet**. Verás un aviso amarillo con tu rama y el botón **Compare & pull request**. Haz clic.
2. **Título:** el mismo del issue, por ejemplo *SN-01: Diseñar el modelo entidad-relación*.
3. **Descripción:** resume qué hiciste y agrega la línea **Closes #N** (donde N es el número del issue en GitHub). Así el issue se cierra solo cuando el PR se aprueba.
4. En **Reviewers** elige a un compañero para que revise. Haz clic en **Create pull request**.
5. Si el revisor pide cambios, corrígelos en la misma rama, haz commit y push de nuevo — el PR se actualiza solo.
6. Cuando esté aprobado, se hace clic en **Merge pull request** y luego **Delete branch**.

## 1.4 Reglas del equipo

- Nunca hacer **git push** directo a **main**.
- Nunca subir **appsettings.Development.json** (contiene tu contraseña de MySQL).
- Un issue = una rama = un Pull Request.
- Hacer **git pull** en main antes de crear cada rama nueva, para no trabajar sobre código viejo.
- Si al hacer pull aparece un **conflicto**, no lo resuelvas a ciegas: avisa en el grupo.

## 2. Tus tareas del Sprint 1

Tienes 4 issues encadenados: el diagrama (SN-01) alimenta las clases C# (SN-02) y el script SQL (SN-04); el DbContext (SN-03) conecta ambos. Total: **10 puntos**.

### SN-01 — Diseñar el modelo entidad-relación de ShipNet

|                      |                          |
|----------------------|--------------------------|
| <b>Sprint</b>        | Sprint 1 (11/09 - 15/09) |
| <b>Puntos</b>        | 3                        |
| <b>Rama sugerida</b> | feature/sn-01-...        |

#### Pasos tentativos

1. Lista las entidades del sistema. Las mínimas son: **Estudiante, Docente, Materia, Grupo, Equipo, SesionAsistencia, RegistroAsistencia, Evaluacion, IntentoEvaluacion**.
2. Define los campos de cada una. Guía de referencia:  
Estudiante: Id, Nombre, Apellido, CI, ApplicationUserId  
Docente: Id, Nombre, Apellido, ApplicationUserId  
Materia: Id, Nombre, Codigo  
Grupo: Id, Nombre, MateriaId, DocenteId  
Equipo: Id, Codigo (ej. "PC-14"), MacAddress, Ubicacion  
SesionAsistencia: Id, GrupoId, Fecha, HoraInicio, HoraFin, Abierta  
RegistroAsistencia: Id, SesionAsistenciaId, EstudianteId, EquipoId, HoraRegistro, Estado (Presente/Tarde/Ausente)  
Evaluacion: Id, GrupoId, Titulo, FechaInicio, FechaFin, DuracionMinutos  
IntentoEvaluacion: Id, EvaluacionId, EstudianteId, EquipoId, FechaInicio, FechaFin, Puntaje
3. Define las relaciones: Materia 1-N Grupo · Docente 1-N Grupo · Grupo 1-N SesionAsistencia · SesionAsistencia 1-N RegistroAsistencia · Estudiante 1-N RegistroAsistencia · Equipo 1-N RegistroAsistencia · Grupo 1-N Evaluacion · Evaluacion 1-N IntentoEvaluacion.
4. Dibuja el diagrama en **dbdiagram.io** (recomendado, exporta a SQL) o draw.io.
5. Compártelo en el grupo para validarlo antes de pasar a SN-02, así no rehaces código después.
6. Guarda la imagen/exportación en la carpeta **Database/** del repo.

**ApplicationUserId** en Estudiante y Docente apunta a la tabla de usuarios de Identity (AspNetUsers) que configura Baruc en SN-06. Coordina con él el tipo de esa columna: Identity usa **VARCHAR(255)** por defecto para el Id de usuario.

#### Criterio de aceptación

- [ ] Diagrama ER entregado y validado por el equipo
- [ ] Todas las relaciones y claves foráneas definidas

### SN-02 — Crear los modelos de EF Core en Models/

|                      |                          |
|----------------------|--------------------------|
| <b>Sprint</b>        | Sprint 1 (11/09 - 15/09) |
| <b>Puntos</b>        | 3                        |
| <b>Rama sugerida</b> | feature/sn-02-...        |

#### Pasos tentativos

1. Crea un archivo por entidad en **Models/** (ej. **Models/Equipo.cs**), namespace **shipnet.Models**.
2. Sigue este patrón para cada clase (ejemplo con dos entidades relacionadas):

```
namespace shipnet.Models;

public class Equipo
{
    public int Id { get; set; }
    public stringCodigo { get; set; } = string.Empty;
    public string MacAddress { get; set; } = string.Empty;
    public string Ubicacion { get; set; } = string.Empty;

    public ICollection<RegistroAsistencia> Registros { get; set; }
        = new List<RegistroAsistencia>();
}

public class RegistroAsistencia
{
    public int Id { get; set; }

    public int SesionAsistenciaId { get; set; }
    public SesionAsistencia SesionAsistencia { get; set; } = null!;

    public int EstudianteId { get; set; }
    public Estudiante Estudiante { get; set; } = null!;

    public int EquipoId { get; set; }
    public Equipo Equipo { get; set; } = null!;

    public DateTime HoraRegistro { get; set; }
    public string Estado { get; set; } = "Presente";
}
```

3. Para cada relación 1-N: en el lado "1" una **ICollection<T>**, en el lado "N" la propiedad **XxxId** más la propiedad de navegación **Xxx**.
4. Los nombres de clase y de propiedad deben coincidir **exactamente** con los nombres de tabla y columna que crearás en SN-04, porque no se usan migraciones.
5. Compila con **dotnet build** para confirmar que no hay errores.

#### Criterio de aceptación

- [ ] Una clase C# por entidad del diagrama
- [ ] Propiedades de navegación definidas en ambos lados de cada relación
- [ ] dotnet build sin errores

## SN-03 — Configurar ApplicationDbContext con Pomelo MySQL

|                      |                          |
|----------------------|--------------------------|
| <b>Sprint</b>        | Sprint 1 (11/09 - 15/09) |
| <b>Puntos</b>        | 2                        |
| <b>Rama sugerida</b> | feature/sn-03-...        |

#### Pasos tentativos

1. Verifica que el paquete esté instalado (Angel lo agrega en SN-00; si no está, instálalo):

```
dotnet add package Pomelo.EntityFrameworkCore.MySql --version 8.0.2
```

2. Crea **Data/ApplicationDbContext.cs**. Hereda de **IdentityDbContext<ApplicationUser>** (la clase ApplicationUser la crea Baruc en SN-06 — si aún no existe, hereda de **DbContext** temporalmente y cámbialo cuando esté):

```
using Microsoft.AspNetCore.Identity.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore;
using shipnet.Models;

namespace shipnet.Data;

public class ApplicationDbContext : IdentityDbContext<ApplicationUser>
{
    public ApplicationDbContext(DbContextOptions<ApplicationDbContext> options)
        : base(options) { }

    public DbSet<Estudiante> Estudiantes => Set<Estudiante>();
    public DbSet<Docente> Docentes => Set<Docente>();
    public DbSet<Materia> Materias => Set<Materia>();
    public DbSet<Grupo> Grupos => Set<Grupo>();
    public DbSet<Equipo> Equipos => Set<Equipo>();
    public DbSet<SesionAsistencia> SesionesAsistencia => Set<SesionAsistencia>();
    public DbSet<RegistroAsistencia> RegistrosAsistencia => Set<RegistroAsistencia>();
    public DbSet<Evaluacion> Evaluaciones => Set<Evaluacion>();
    public DbSet<IntentoEvaluacion> IntentosEvaluacion => Set<IntentoEvaluacion>();
}
```

3. Registra el DbContext en **Program.cs**, antes de **builder.Build()**:

```
var connectionString = builder.Configuration.GetConnectionString("Default");
builder.Services.AddDbContext<ApplicationDbContext>(options =>
    options.UseMySQL(connectionString, ServerVersion.AutoDetect(connectionString)));
```

4. La cadena de conexión ya está en tu **appsettings.Development.json** local (la copiaste en la sección 1.1). Confirma que la base **shipnet** exista en tu MySQL (la creas en SN-04).
5. Corre **dotnet run**: si la app arranca sin excepción de conexión, está listo.

**No corras** dotnet ef migrations ni Database.EnsureCreated(): el esquema lo creas tú con SQL en SN-04. EF Core solo lee y escribe sobre tablas que ya existen.

### Criterio de aceptación

- [ ] DbContext con todos los DbSet registrado en Program.cs
- [ ] La app arranca y se conecta a MySQL local sin errores

## SN-04 — Crear script SQL (DDL) de la base de datos en Workbench y subirlo al repo

|                      |                          |
|----------------------|--------------------------|
| <b>Sprint</b>        | Sprint 1 (11/09 - 15/09) |
| <b>Puntos</b>        | 2                        |
| <b>Rama sugerida</b> | feature/sn-04-...        |

### Pasos tentativos

1. En MySQL Workbench crea la base y las tablas según SN-01. Respeta el orden: primero las tablas sin claves foráneas (Materia, Equipo, Docente...), después las que dependen de ellas.
2. Ejemplo de dos tablas con su relación:

```
CREATE DATABASE IF NOT EXISTS shipnet
CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
```

```
USE shipnet;
```

```
CREATE TABLE Equipos (  
    Id            INT AUTO_INCREMENT PRIMARY KEY,  
    Codigo        VARCHAR(20) NOT NULL,  
    MacAddress    VARCHAR(17) NOT NULL UNIQUE,  
    Ubicacion     VARCHAR(100) NULL  
);
```

```
CREATE TABLE SesionesAsistencia (  
    Id            INT AUTO_INCREMENT PRIMARY KEY,  
    GrupoId       INT NOT NULL,  
    Fecha         DATE NOT NULL,  
    HoraInicio    TIME NULL,  
    HoraFin       TIME NULL,  
    Abierta       TINYINT(1) NOT NULL DEFAULT 1,  
    CONSTRAINT FK_Sesiones_Grupo FOREIGN KEY (GrupoId) REFERENCES Grupos(Id)  
);
```

3. Los nombres de tabla deben coincidir con los **DbSet** de SN-03 (Equipos, Grupos, SesionesAsistencia...) y las columnas con las propiedades de SN-02.
4. Las tablas de Identity (AspNetUsers, AspNetRoles, AspNetUserRoles, etc.) también van en este script, ya que no hay migraciones. Coordina con Baruc: él puede generarte el SQL exacto de esas tablas desde su tarea SN-06.
5. Exporta todo a un único archivo **Database/schema.sql** (Workbench: Server > Data Export, solo estructura, sin datos) y súbelo en tu PR.
6. Prueba el script en una base limpia: borra **shipnet** y vuelve a correr el archivo completo. Si corre de principio a fin sin errores, cualquiera del equipo podrá recrear la base.

### Criterio de aceptación

- [ ] Database/schema.sql en el repo
- [ ] El script corre completo sobre una base vacía sin errores
- [ ] Nombres de tablas y columnas coinciden con los modelos C#